From Jollof to DataFrames

A friendly tour of data types, datasets, and Python structures

Today's kitchen rule: Before you cook, you learn your ingredients.
Before you analyse data, you learn what *kind* of thing each value is, and how the table is organised.

Imagine you run a small food stall that sells Cameroonian and Nigerian favourites: jollof rice, ndolé, suya, puff-puff, egusi soup, plantain... Customers come every day. You write things down. That notebook of sales? That's a dataset. This lesson turns that kitchen notebook into the language of data science.

Learning goals

By the end of this lesson, you should be able to:

1. Open a Jupyter notebook and run cells with confidence
2. Explain what a dataset, row, column, variable, and value mean
3. Recognise common data types (`int`, `float`, `str`, `bool`, missing)
4. Build small tables using lists and dicts
5. Spot type mistakes that will break next week's data manipulation

---

1. Warm-up: your Jupyter kitchen

Think of Jupyter as a shared cooking counter:

Jupyter idea	Kitchen analogy
Markdown cell	Recipe notes and explanations
Code cell	The actual cooking steps
Run cell	“Fire!” — execute that step
Kernel restart	Wipe the counter clean and start fresh
Run order	You can’t fry plantain before you peel it

Tips that save headaches:

- Run cells from top to bottom when learning
- If something “mysteriously” fails, restart the kernel and re-run from the top
- Use `print(...)` freely — tasting the soup as you cook

```
None
print("Welcome to Mama's Jollof & Ndolé Kitchen!")
```

---

2. What is a dataset? (Your stall’s sales book)

A dataset is organised information, usually a table with:

- Rows → one observation (one sale, one customer order, one day)
- Columns → one measured thing (price, dish name, city, spicy?)

Same idea, many names

People mix these words. In this class, treat them as cousins:

Word you hear	What it usually means
dataset / table / dataframe	The whole table
row / observation / record / example	One line in the table
column / variable / attribute / feature	One field measured for every row
value / cell / entry	What's inside one row–column meeting point

Memory hook:
Row = one plate of food.
Column = one question you ask about every plate (name? price? spicy?).

Example: weekend sales at the stall

order_id	dish	country_style	city	price_xaf	portions	is_spicy	sold_out
1	Jollof Rice	Nigerian	Douala	1500	2	True	False
2	Ndolé	Cameroonian	Yaoundé	2000	1	False	False
3	Suya	Nigerian	Lagos	2500	3	True	True

4	Eru & Water Fufu	Cameroonian	Bamenda	1800	2	False	False
5	Egusi Soup	Nigerian	Abuja	2200	1	True	False
6	Puff-Puff (6 pcs)	Nigerian	Douala	500	1	False	True
7	Fried Plantain	Cameroonian	Limbe	800	2	False	False
8	Pepper Soup	Nigerian	Port Harcourt	3000	1	True	False

Quick check (say it out loud):

- How many observations (rows) are there? → 8
- What are some variables (columns)? → `dish`, `price_xaf`, `is_spicy`, ...
- What is the value of `dish` for order 3? → `"Suya"`

---

3. Data types: what *kind* of ingredient is this?

In cooking, salt is not the same as palm oil.
In Python, `1500` is not the same as `"1500"`.

3.1 The types you need this week

Kind of information	Real-world example	Python type	Looks like
Whole number	portions sold = 3	int	3
Decimal number	spice rating = 4.5	float	4.5
Text / category	dish name	str	"Ndolé"
Yes / No	is it spicy?	bool	True / False
Missing / unknown	customer didn't say city	None (for now)	None

Next week, when we use pandas, missing numbers often show up as NaN ("Not a Number"). Same idea: *we don't have that value.*

3.2 Meet them in Python

```
None
portions = 3            # int  — whole plates of jollof
price_xaf = 1500        # int  — price in Central African CFA francs
spice_level = 4.5       # float — half steps allowed
dish = "Jollof Rice"    # str  — always in quotes
is_spicy = True         # bool  — True or False (capital T/F)
customer_note = None    # NoneType — missing for now

print(type(portions))
print(type(price_xaf))
print(type(spice_level))
print(type(dish))
print(type(is_spicy))
```

```
print(type(customer_note))
```

3.3 The classic kitchen trap: "1500" vs 1500

```
None
price_number = 1500
price_text = "1500"

print(type(price_number)) # <class 'int'>
print(type(price_text))   # <class 'str'>

print(price_number + 500) # 2000 ✅ makes sense (add tip / extra meat)
# print(price_text + 500) # ❌ TypeError — you can't add text + number
print(price_text + " XAF") # "1500 XAF" ✅ text joins text
```

Why this matters for data science:
If a CSV file stores prices as text, your "average price of suya" will fail or look nonsense until you fix the type.

3.4 Categories vs numbers (thinking like an analyst)

Not every number-looking thing should be treated as math:

Column	Values	Better treated as...	Why
price_xaf	500, 1500, 3000	numeric	You sum/average them
portions	1, 2, 3	numeric	You count and total
city	Douala, Lagos	categorical / text	Labels, not math

country_style	Cameroonian, Nigerian	categorical	Groups for comparison
order_id	1, 2, 3...	ID / label (often)	Identifies a row; averaging IDs is silly
is_spicy	True / False	boolean	Filters: “only spicy dishes”

Story prompt:
Is “Cameroonian vs Nigerian jollof” a number? No — it’s a category.
Is “how many portions of eru sold on Saturday” a number? Yes — that’s numeric.

---

4. Data structures: bowls to hold your ingredients

Python needs containers before pandas arrives. Master these two first.

4.1 List — an ordered queue of plates

A list keeps items in order, like orders waiting at the counter.

```
None
todays_dishes = ["Jollof Rice", "Ndolé", "Suya", "Egusi Soup", "Puff-Puff"]

print(todays_dishes[0])  # first item → "Jollof Rice"
print(todays_dishes[-1]) # last item  → "Puff-Puff"
print(len(todays_dishes)) # how many dishes on the board
```

Lists can hold mixed types (useful, but be careful):

```
None
suya_order = ["Suya", 2500, 3, True]  # dish, price, portions, is_spicy
```

4.2 Dict — labelled containers (name → value)

A dictionary stores keys and values — like a labelled spice jar.

```
None
ndole_plate = {
   "dish": "Ndolé",
   "country_style": "Cameroonian",
   "city": "Yaoundé",
   "price_xaf": 2000,
   "portions": 1,
   "is_spicy": False,
}

print(ndole_plate["dish"])
print(ndole_plate["price_xaf"])
```

Bridge to next week:
A pandas DataFrame feels like a dict of columns, where each column name points to a list of values.

4.3 Building a tiny "table" with lists + a dict

This is the mental model of a spreadsheet — without pandas yet.

```
None
sales = {
   "dish": [
      "Jollof Rice",
      "Ndolé",
      "Suya",
      "Eru & Water Fufu",
      "Egusi Soup",
      "Puff-Puff",
      "Fried Plantain",
      "Pepper Soup",
```

```
    ],
    "country_style": [
        "Nigerian",
        "Cameroonian",
        "Nigerian",
        "Cameroonian",
        "Nigerian",
        "Nigerian",
        "Cameroonian",
        "Nigerian",
    ],
    "city": [
        "Douala",
        "Yaoundé",
        "Lagos",
        "Bamenda",
        "Abuja",
        "Douala",
        "Limbe",
        "Port Harcourt",
    ],
    "price_xaf": [1500, 2000, 2500, 1800, 2200, 500, 800, 3000],
    "portions": [2, 1, 3, 2, 1, 1, 2, 1],
    "is_spicy": [True, False, True, False, True, False, False, True],
}

# Column = one variable
print(sales["dish"])

# One observation (row 0) — gather across columns
row0 = {col: sales[col][0] for col in sales}
print(row0)
```

Check yourself:

- `sales["price_xaf"]` → a column (all prices)
- index `2` across columns → the Suya order (one row)

---

5. Guided practice (do this in a notebook)

Copy these into cells. Taste as you go.

Exercise A — Name the parts

Using the stall table above, write short answers in a markdown cell:

1. One observation
2. Three variables
3. One numeric column and one categorical column
4. The value of `city` for the Eru order

Exercise B — Types detective

```
None
values = [
    1500,
    "1500",
    4.5,
    "Ndolé",
    True,
    None,
    "Lagos",
    0,
]

for v in values:
    print(v, "→", type(v))
```

Predict *before* you run. Were you right about `"1500"` and `0`?

Exercise C — Broken jollof math (on purpose)

```
None
prices = [1500, 2000, "2500", 1800]  # oops: suya price typed as text
```

```
# total = sum(prices)  # try it — what happens?
```

Fix it two ways students often use later:

```
None
# 1) Don't put quotes around numbers when you mean quantities
prices_fixed = [1500, 2000, 2500, 1800]
print(sum(prices_fixed))

# 2) Convert text that *should* be numeric
prices_messy = [1500, 2000, "2500", 1800]
prices_clean = [int(p) for p in prices_messy]
print(sum(prices_clean))
```

Exercise D — Mini menu as a dict-of-lists

Create your own 5-row dataset with at least:

- 1 Nigerian dish and 1 Cameroonian dish you love
- columns: dish, country_style, price_xaf, is_spicy
- print the type of one price and one dish name

---

6. Cameroon × Nigeria flavour bank (use these in class!)

Steal freely for examples, quizzes, and homework:

Dish	Style	Fun teaching angle

Jollof Rice	Nigerian (friendly rivalry welcome)	Category: style; Numeric: pot size / price
Ndolé	Cameroonian	Bitterness level as float? or spicy bool?
Eru / Okok	Cameroonian	Portions of water fufu as `int`
Achu / Yellow Soup	Cameroonian	City: Northwest — categorical
Suya	Nigerian	`is_spicy=True` almost always
Egusi Soup	Nigerian	Pair with pounded yam — two columns
Puff-Puff	Nigerian snack	Count pieces (`int`), sugar level (`float`)
Fried / Roasted Plantain	Shared favourite	Price varies by city — great for later groupby
Pepper Soup	Nigerian	Boolean: `has_fish`, `has_goat`
Kokki Beans / Beans & Stew	Everyday hero	Missing data: customer forgot portion size → `None`

Classroom debate (2 minutes, high energy):
Who makes better jollof — Nigeria or Cameroon?
Then pivot: "That's a categorical opinion, not a column we can average... unless we collect ratings as numbers!"

---

7. Checklist before next week (data manipulation)

Tick these off:

- [ ] I can run notebook cells top-to-bottom
- [ ] I can explain row vs column using a food table
- [ ] I know `int`, `float`, `str`, `bool`, and `None`
- [ ] I understand why `"2500"` is dangerous in a price column
- [ ] I can build a tiny table with a dict of lists
- [ ] I'm ready to load a CSV and inspect column types next class

Next week preview

We will:

1. Load a food-sales CSV into pandas
2. Inspect `head()`, `shape`, and `dtypes`
3. Select columns, filter rows ("only spicy Nigerian dishes in Douala")
4. Fix messy types before calculating totals

---

8. Glossary (keep this open while coding)

Term	Simple meaning
Dataset	Organised data, often a table
Observation / row	One example (one order)

Variable / column / attribute / feature	One measured field
Data type	The kind of value (number, text, true/false…)
List	Ordered collection: `[...]`
Dict	Key → value mapping: `{...}`
Missing value	Unknown / not recorded (`None` now; NaN later)
Categorical	Labels/groups (city, dish style)
Numeric	Values you can do math with

---

9. Exit ticket (2 minutes)

Answer in one sentence each:

1. In your own words: what is a variable in a dataset?
2. Give one Cameroonian example of a categorical value and one Nigerian example of a numeric value.
3. Why might `sum(["1500", "2000"])` fail or surprise you?

Closing line for class

Great analysts don’t start by “running fancy code.”
They start by knowing whether they’re looking at jollof portion counts… or jollof names written as text.
Master the ingredients this week — next week, we cook.

See you next class — bring curiosity, and maybe an appetite. 🍲